🧩 Funciones en Python

def · parámetros · return · ámbito · argumentos por posición, por nombre y por defecto · *args

Lógica de Programación 1 · Guía del estudiante

Desde la primera clase de Python usas funciones que otros escribieron: `print()` muestra, `input()` lee, `int()` convierte, `range()` genera números, `len()` mide. Las usas sin saber cómo están hechas por dentro: solo sabes **qué reciben** y **qué devuelven**. Una **función** es un bloque de código con nombre que realiza una tarea específica: recibe datos (sus **parámetros**), los procesa y puede **devolver** un resultado con `return`.

1 · ¿Por qué crear funciones?

Ventaja	Qué significa
Modularidad	divides un programa grande en piezas pequeñas que puedes probar por separado.
Reutilización	escribes el código una vez y lo usas las veces que quieras, con datos distintos.
Abstracción	quien la usa solo necesita su nombre, sus entradas y su salida (como haces con `print()`).

con_funcion.py

```
def linea():
    print("—" * 30)

linea()
print("  FACTURA ELECTRÓNICA")
linea()
print("  Cliente: Laura Restrepo")
print("  Total: $48.500")
linea()
```

```
——————————————————————————
  FACTURA ELECTRÓNICA
——————————————————————————
  Cliente: Laura Restrepo
  Total: $48.500
——————————————————————————
```

Si quieres cambiar el estilo de la línea, lo cambias en un solo lugar: dentro de `linea()`.

2 · Definir y llamar una función

bienvenida.py

```
def bienvenida(nombre):
    mensaje = f"¡Hola, {nombre}! Bienvenido al curso"
    return mensaje

texto = bienvenida("Laura")
print(texto)
print(bienvenida("Mateo"))
```

```
¡Hola, Laura! Bienvenido al curso
¡Hola, Mateo! Bienvenido al curso
```

Parte	Significado
`def`	palabra reservada: "defino una función"
`bienvenida`	el nombre: un verbo en snake_case
`(nombre)`	los parámetros: sus entradas (cero o varios)
`:` y sangría	el cuerpo: lo que hace la función
`return`	entrega el resultado a quien la llamó

¿Qué pasa al llamarla? (1) Python lee el `def` y **guarda** la función sin ejecutar su cuerpo. (2) En `bienvenida("Laura")` el programa **salta** a la función y el parámetro `nombre` vale `"Laura"`. (3) Se ejecuta el cuerpo y `return` devuelve el mensaje. (4) El programa **regresa** a donde estaba y sigue.

Error frecuente	Ejemplo	Qué pasa
Llamar antes de definir	`calcular(5)` y luego `def calcular(x):`	**NameError**: define primero, llama después.
Olvidar los paréntesis	`print(bienvenida)`	no se ejecuta: muestra `<function bienvenida at 0x…>`.
Faltan argumentos	`bienvenida()`	**TypeError**: missing 1 required positional argument.
Sobran argumentos	`bienvenida("Ana", "Luis")`	**TypeError**: takes 1 positional argument but 2 were given.

3 · Parámetros y return

Un **parámetro** es la variable que aparece en la definición (`def area(base, altura):`); un **argumento** es el valor que se envía en la llamada (`area(5, 3)`) y se copia en el parámetro de la misma posición. `return` entrega un valor y **termina** la función de inmediato; sin return, la función devuelve `None`.

precio.py

```
def precio_final(precio, descuento):
    con_descuento = precio * (1 - descuento / 100)
    return con_descuento * 1.19

total = precio_final(100000, 20)
print(f"Pagas: {total:.2f}")
print(f"Dos unidades: {2 * precio_final(100000,
20):.2f}")
```

```
Pagas: 95200.00
Dos unidades: 190400.00
```

Lo devuelto se puede guardar en una variable, mostrar o usar dentro de otra operación.

¿print o return?

printreturn.py

```
def doble_print(x):
    print(x * 2)

def doble_return(x):
    return x * 2

a = doble_print(5)
b = doble_return(5)
print("a =", a, "| b =", b)
```

```
10
a = None | b = 10
```

print le muestra algo a la persona; **return** le entrega un valor al programa. Una función que solo hace print devuelve `None`: no puedes sumarlo, compararlo ni guardarlo.

Funciones que devuelven True o False

votar.py

```
def es_mayor_de_edad(edad):
    return edad >= 18

for edad in [15, 18, 32, 17]:
    if es_mayor_de_edad(edad):
        print(edad, "→ puede votar")
    else:
        print(edad, "→ aún no")
```

```
15 → aún no
18 → puede votar
32 → puede votar
17 → aún no
```

Una comparación ya es True o False: se puede devolver directamente y usar la función como condición de un if.

4 · Ámbito de las variables

Local	Global
se crea **dentro** de la función (incluidos sus parámetros); solo existe mientras la función se ejecuta.	se crea **fuera** de las funciones; las funciones pueden **leerla**.

ambito.py

```
tasa_iva = 0.19  # global

def con_iva(precio):
    total = precio * (1 + tasa_iva)  # total es local
    return total

print(con_iva(1000))
print(total)  # ¿existe total aquí afuera?
```

```
1190.0
NameError: name 'total' is not
defined
```

sombra.py

```
puntos = 10

def jugar():
    puntos = 99
    print("Dentro:", puntos)

jugar()
print("Fuera:", puntos)
```

```
Dentro: 99
Fuera: 10
```

`puntos = 99` crea **otra** variable, local: la global no cambia. Si una función necesita un dato, recíbelo como **parámetro**; si produce un resultado, entrégalo con **return**.

5 · Formas de pasar argumentos

pedido.py

```
def pedido(producto, cantidad, precio, envio=5000):
    total = cantidad * precio + envio
    return f"{cantidad} x {producto}: ${total}"

print(pedido("empanada", 6, 2500))
print(pedido("arepa", 3, 4000, envio=0))
print(pedido(precio=12000, producto="pizza",
cantidad=2))
```

```
6 x empanada: $20000
3 x arepa: $12000
2 x pizza: $29000
```

Forma	Ejemplo	Regla
Posicional	`pedido("arepa", 3, 4000)`	cada valor va al parámetro de la misma posición: el orden importa
Por nombre	`pedido(cantidad=3, producto="arepa", precio=4000)`	el orden no importa; el nombre debe existir (si no, **TypeError**)
Por defecto	`def pedido(…, envio=5000):`	si no se envía, usa ese valor; van **al final** de la definición (si no, **SyntaxError**)
Combinadas	`pedido("arepa", 3, 4000, envio=0)`	al llamar: primero los posicionales, después los nombrados

args.py — *args: cualquier cantidad de argumentos

```
def promedio(*notas):
    if len(notas) == 0:
        return 0
    suma = 0
    for n in notas:
        suma += n
    return suma / len(notas)

print(round(promedio(4.5, 3.0, 5.0), 2))
print(round(promedio(2.8, 3.6), 2))
print(promedio())
```

```
4.17
3.2
0
```

`*notas` agrupa todos los valores recibidos y se recorre con un for. Así funciona `print()`. Existe también `**kwargs` para argumentos por nombre (lo verás con los diccionarios).

6 · Practica sin computador

¿Qué imprime?

A

```
def g(n):
    n = n * 2
print(g(5))
```

Salida: _____

B

```
def suma(a, b=10):
    return a + b
print(suma(5), suma(5, 1))
```

Salida: _____

C

```
x = 5
def f(x):
    x = x + 1
    return x
y = f(x)
print(x, y)
```

Salida: _____

Conecta los argumentos: ¿qué valor recibe cada parámetro?

`def pedido(producto, cantidad, precio, envio=5000):`

Llamada	producto	cantidad	precio	envio
`pedido("pizza", 2, 12000, 0)`				
`pedido(precio=2500, producto="empanada", cantidad=6)`				
`pedido("jugo", cantidad=4, precio=3500)`				

Resumen para tomarle foto 📸

Definir
`def nombre(parametros):`

Llamar
`x = nombre(argumentos)`

Devolver
`return valor` (sin return → None)

Ámbito
local: dentro · global: fuera

Por nombre y por defecto
`f(b=2, a=1)` · `def f(a, b=2):`

Cantidad variable
`def f(*args):`

7 · Caja de herramientas para los ejercicios

Quiero...	En Python
usar lo que devuelve una función	`x = f(a, b)` · `print(f(a, b))` · `if es_algo(x):`
una función que usa otra	`b = bruto(h, v)` y luego `return b - deducciones(b)`
salir apenas sepa la respuesta	`return` dentro de un if o de un for
potencia (también negativa)	`(1 + i) ** (-n)`
cociente y residuo	`monto // 20000` · `monto % 20000`
2 decimales	`f"{x:.2f}"`

En cada ejercicio decide primero: ¿qué funciones necesito?, ¿qué recibe cada una y qué devuelve?, ¿qué hace el programa principal? Usa **exactamente** los nombres de funciones del enunciado: en el 🐍 **Laboratorio** de la lección interactiva el corrector llama directamente a tus funciones para revisar lo que devuelven. Escribe y prueba primero la función más pequeña.

8 · Ejercicios propuestos

12 ejercicios de menor a mayor dificultad. Bosqueja tus funciones y tu programa principal en el espacio.

🟢 Nivel 1 · Calentamiento — Una función que calcula y devuelve, y un programa principal que la usa.

Nivel 1 · def + return + for

1. Precios con IVA

Crea la función `precio_con_iva(precio)` que **devuelva** (no que imprima) el precio con el IVA del 19 % incluido. Luego escribe el programa principal: pide cuántos productos hay (N) y el precio sin IVA de cada uno; usando la función, muestra `Producto k: X` para cada uno y al final `Total con IVA: X` (2 decimales).
Entrada: un entero N y N reales · **Salida:** una línea por producto y el total
Ejemplo: 2 productos: 1000 y 2500 → `Producto 1: 1190.00` · `Producto 2: 2975.00` · `Total con IVA: 4165.00`. Además, el corrector llama a tu función directamente: `precio_con_iva(1000)` debe devolver 1190.0.
💡 **Pista:** la función solo calcula y devuelve; los `input` y los `print` van en el programa principal. El total se puede acumular con lo que devuelve la función en cada vuelta.

Nivel 1 · parámetros + return

2. Promedio de tres notas

Crea `promedio(a, b, c)` que reciba tres notas y **devuelva** su promedio. En el programa principal pide las tres notas, usa la función y muestra `Promedio: X` (2 decimales) y `Aprobó` si el promedio es de 3.0 o más, o `Reprobó` si no.

Entrada: tres reales · **Salida:** dos líneas

Ejemplo: 4.0, 3.5 y 4.2 → `Promedio: 3.90` · `Aprobó`. El corrector también prueba `promedio(5, 5, 5)`, que debe devolver 5.0.

💡 **Pista:** los tres valores llegan a la función por sus parámetros, en el mismo orden en que los escribes al llamarla. Guarda lo que devuelve en una variable para poder mostrarlo y decidir con él.

Nivel 1 · return booleano

3. ¿Par o impar?

Crea `es_par(n)` que **devuelva** `True` si n es par y `False` si no. La función **no imprime nada**. En el programa principal pide un entero y usa la función dentro de un `if` para mostrar `N es par` o `N es impar`.

Entrada: un entero · **Salida:** un mensaje

Ejemplo: 7 → `7 es impar`; 12 → `12 es par`. El corrector comprueba que `es_par(4)` devuelva el booleano `True` (no el texto "True").

💡 **Pista:** una comparación ya es un valor True o False: puedes devolverla directamente. Una función que devuelve un booleano se puede escribir tal cual como condición de un if.

🟡 Nivel 2 · Intermedio — Valores por defecto, argumentos por nombre, condiciones y fórmulas financieras.

Nivel 2 · parámetro por defecto + argumentos por nombre

4. Descuento con valor por defecto

Crea `descuento(precio, porcentaje=10)` que devuelva el precio después de aplicar el descuento. Si al llamarla no se da el porcentaje, debe usar **10 %**. En el programa principal pide el precio y el porcentaje y muestra `Precio final: X` (2 decimales).

Entrada: dos reales · **Salida:** una línea

Ejemplo: 80000 y 25 → `Precio final: 60000.00`. El corrector llama `descuento(50000)` (debe usar el 10 %: 45000.0) y `descuento(porcentaje=50, precio=80000)` (40000.0).

💡 **Pista:** el valor por defecto se escribe en la lista de parámetros de la definición; quien llama puede omitir ese argumento o darlo por su nombre en cualquier orden.

Nivel 2 · return + condicionales

5. El mayor de tres

Crea `mayor_de_tres(a, b, c)` que devuelva el mayor de los tres números **sin usar `max()`**. Debe funcionar también cuando hay valores repetidos o negativos. En el programa principal pide tres enteros y muestra `El mayor es X`.

Entrada: tres enteros · **Salida:** una línea

Ejemplo: 4, 9 y 2 → `El mayor es 9`. El corrector prueba, entre otras, `mayor_de_tres(7, 7, 3)` → 7 y `mayor_de_tres(-5, -2, -9)` → −2.

💡 **Pista:** puedes suponer que el primero es el mayor y luego compararlo con los otros dos, o comparar con `and`. Prueba casos con empates: ahí suelen fallar las comparaciones estrictas.

Nivel 2 · return con fórmula financiera

6. Cuota fija de un crédito

Un banco presta con **cuota fija**. Si i es la tasa mensual en decimal (2 % → 0.02), la cuota es `monto × i ÷ (1 − (1 + i)^−meses)`; si la tasa es 0, la cuota es simplemente `monto ÷ meses`. Crea `cuota_mensual(monto, tasa, meses)` (tasa en %) que devuelva la cuota. En el programa principal pide los tres datos y muestra `Cuota: X`, `Total pagado: X` y `Intereses: X` (2 decimales).

Entrada: un real, un real y un entero · **Salida:** tres líneas

Ejemplo: 1000000, 2 % y 12 meses → `Cuota: 94559.60` · `Total pagado: 1134715.16` · `Intereses: 134715.16`.

💡 **Pista:** en Python la potencia es `**` y un exponente negativo es válido. Convierte la tasa de porcentaje a decimal dentro de la función, y no olvides el caso de la tasa 0 (¿qué pasaría con la fórmula?).

🟠 Nivel 3 · Avanzado — Funciones que llaman a otras funciones, ciclos dentro de funciones y menús.

Nivel 3 · funciones que usan funciones + for + while

7. ¿De cuántas formas?

En una heladería hay n sabores y quieres un helado de k sabores distintos. El número de formas de elegirlos es `n! ÷ (k! × (n − k)!)`. Crea `factorial(n)` (con un ciclo y un acumulador, sin `math`) y `combinaciones(n, k)`, que **use** `factorial`. En el programa principal pide n y luego k; mientras k no esté entre 0 y n, muestra `k debe estar entre 0 y n` y vuelve a pedirlo. Muestra `Formas: X` (entero).

Entrada: dos o más enteros · **Salida:** los avisos y el resultado

Ejemplo: n = 5, k = 2 → `Formas: 10`; n = 4, k = 7, luego 2 → `k debe estar entre 0 y n` · `Formas: 6`. El corrector prueba `factorial(0)` → 1 y `factorial(6)` → 720.

💡 **Pista:** 0! vale 1: piensa con qué valor empieza el acumulador y qué rango recorre el ciclo para que ese caso salga solo. Para que el resultado sea entero, la división entre factoriales puede hacerse con `//`.

Nivel 3 · funciones + recorrer texto

8. Detector de palíndromos

Un palíndromo se lee igual al derecho y al revés, sin contar espacios ni mayúsculas ("Anita lava la tina"). Crea `limpiar(frase)` que devuelva la frase en minúsculas y **sin espacios**, y `es_palindromo(frase)` que use `limpiar` y devuelva `True` o `False`. En el programa principal pide una frase y muestra `Es palíndromo` o `No es palíndromo`.

Entrada: un texto · **Salida:** un mensaje

Ejemplo: "Anita lava la tina" → `Es palíndromo`; "Lógica" → `No es palíndromo`. El corrector prueba `limpiar("Ana Mar")` → `"anamar"`.

💡 **Pista:** para limpiar, recorre la frase carácter por carácter y construye un texto nuevo solo con los que no son espacio. Para comparar, puedes recorrer desde los dos extremos hacia el centro, o construir la versión invertida.

Nivel 3 · funciones + menú con while

9. Conversor de unidades

Crea tres funciones que devuelvan la conversión: `c_a_f(c)` (°C a °F: multiplica por 9/5 y suma 32), `km_a_millas(km)` (1 km = 0.621371 millas) y `kg_a_libras(kg)` (1 kg = 2.20462 libras). El programa muestra un menú que se repite hasta elegir salir: **1** °C→°F, **2** km→millas, **3** kg→libras, **4** Salir. Cada conversión pide el valor y muestra `Resultado: X` (2 decimales); otra opción muestra `Opción no válida`. Al salir muestra `Conversiones realizadas: N`.

Entrada: opciones del menú y sus valores · **Salida:** un resultado por conversión y el conteo final

Ejemplo: 1 → 100, 3 → 50, 4 → `Resultado: 212.00` · `Resultado: 110.23` · `Conversiones realizadas: 2`.

💡 **Pista:** cada función hace un solo cálculo y lo devuelve; el menú (ciclo, lectura de la opción, mensajes y contador) va en el programa principal. ¿Cómo se escribe un ciclo que solo termina cuando el usuario lo decide?

🔴 Nivel 4 · Reto ★ — Programas divididos en varias funciones que trabajan juntas.

Nivel 4 · varias funciones + ciclo

★ 10. Nómina modular

Divide la liquidación de una nómina en funciones: `salario_bruto(horas, valor_hora)` paga normal las primeras **48** horas y cada hora adicional con un **recargo del 25 %**; `deducciones(bruto)` devuelve el **8 %** (salud y pensión); `neto(horas, valor_hora)` usa las dos anteriores. El programa pide el valor de la hora (real), la cantidad de empleados y, para cada uno, su nombre y sus horas (entero); muestra `Nombre: neto X` y al final `Total nómina: X` (2 decimales).
Entrada: un real, un entero y, por empleado, un texto y un entero · **Salida:** una línea por empleado y el total
Ejemplo: valor hora 10000; Ana 40 h, Luis 50 h → `Ana: neto 368000.00` · `Luis: neto 464600.00` · `Total nómina: 832600.00`. El corrector prueba `salario_bruto(50, 10000)` → 505000.0.
💡 **Pista:** cada función hace una sola cosa y las más grandes se construyen llamando a las pequeñas. Si cambiara el porcentaje de deducción, ¿en cuántos lugares tendrías que corregir tu programa?

Nivel 4 · funciones + // y % + validación

★ 11. Cajero que entrega billetes

Un cajero entrega la **menor cantidad de billetes** posible con las denominaciones 100000, 50000, 20000, 10000, 5000 y 2000 (en ese orden); lo que no alcanza para un billete de 2000 se entrega en monedas. Crea `cantidad_billetes(monto, denominacion)`, que devuelva cuántos billetes de esa denominación caben en el monto, y `desglosar(monto)`, que muestre una línea `C x D` por cada denominación usada (solo si C > 0) y **devuelva** el total de billetes. El programa pide el monto y, mientras no sea positivo y múltiplo de 1000, muestra `Monto inválido` y lo vuelve a pedir; al final muestra `Total de billetes: N` y `Resto en monedas: R`.
Entrada: uno o más enteros · **Salida:** el desglose, el total y el resto
Ejemplo: 370000 → `3 x 100000` · `1 x 50000` · `1 x 20000` · `Total de billetes: 5` · `Resto en monedas: 0`.
💡 **Pista:** con la división entera sabes cuántos billetes caben y con el residuo, cuánto falta por entregar. El resto en monedas es lo que queda después de la última denominación (el programa principal puede calcularlo aplicando el residuo con cada denominación, en orden).

Nivel 4 · funciones encadenadas + condiciones + ciclo

★ 12. ¿Qué día del año es?

Un año es **bisiesto** si es divisible entre 4 pero no entre 100, o si es divisible entre 400. Crea: `es_bisiesto(anio)` → True/False; `dias_mes(mes, anio)` → días de ese mes (febrero tiene 29 en bisiesto); `fecha_valida(dia, mes, anio)` → True/False; y `dia_del_anio(dia, mes, anio)` → el número de día dentro del año (1 de enero = 1), sumando con un ciclo los días de los meses anteriores. El programa pide día, mes y año: si la fecha no es válida muestra `Fecha inválida`; si lo es, `Día N del año` y `Faltan X días para terminar el año`.

Entrada: tres enteros · **Salida:** dos líneas o el aviso

Ejemplo: 1, 3, 2025 → `Día 60 del año` · `Faltan 305 días para terminar el año`; 29, 2, 2023 → `Fecha inválida`. El corrector prueba `es_bisiesto(1900)` → False y `es_bisiesto(2000)` → True.

💡 **Pista:** construye de abajo hacia arriba: primero la función más pequeña (bisiesto), y prueba cada una antes de usarla en la siguiente. Para los días de cada mes, agrupa los meses que tienen la misma cantidad.